Insertion Sort

Think **"Playing Card Organization"**!

- 🃏 **PICK** the next unsorted card (element)
- 👀 **SCAN BACKWARDS** through your sorted hand
- 🔄 **SHIFT** cards right to make space
- 📍 **INSERT** the card in its perfect spot

Insertion Sort - Philosophy

One at a time, perfectly placed, just like organizing my card hand!

🎴 **PICK** the next unsorted element (like drawing a card)

👀 **SCAN BACKWARDS through the sorted portion (checking your hand)**

🔄 **SHIFT elements right to make space (moving cards over)**

📍 **INSERT the element in its perfect spot (placing the card)**

✅ **SORTED PORTION GROWS by one each round**

Insertion Sort – Strategy

[5, 2, 4, 6, 1, 3]

Pick up a card, find its spot, slide others over, place perfectly!

Round 1: Picks card “2”

```
Hand: [5 | 2, 4, 6, 1, 3]
      ↑    ↑
   sorted  picking up

    "I have 5 in my sorted hand. Where does 2 go?"
👀 SCAN: 5 > 2? Yes! Need to shift 5 right
🔄 SHIFT: [_, 5 | 4, 6, 1, 3]
📍 INSERT: [2, 5 | 4, 6, 1, 3]

Result: [2, 5 | 4, 6, 1, 3]
        ✅✅   ←(sorted portion grew!)
```

Round 2: Picks card “4”

```
Hand: [2, 5 | 4, 6, 1, 3]
          ↑
       picking up

    "Where does 4 fit between 2 and 5?"
👀 SCAN: 5 > 4? Yes, shift 5! 2 > 4? No, found spot!
🔄 SHIFT: [2, _, 5 | 6, 1, 3]
📍 INSERT: [2, 4, 5 | 6, 1, 3]

Result: [2, 4, 5 | 6, 1, 3]
        ✅✅✅   ←(sorted portion grew!)
```

Insertion Sort – Strategy

[5, 2, 4, 6, 1, 3]

Pick up a card, find its spot, slide others over, place perfectly!

Round 3: Picks card “6”

```
Hand: [2, 4, 5 | 6, 1, 3]
              ↑
          picking up


     "6 is bigger than 5, perfect spot!"
👀 SCAN: 5 > 6? No! Already in perfect position!
✅ NO SHIFT NEEDED!

Result: [2, 4, 5, 6 | 1, 3]
        ✅✅✅✅   ←(lucky no-shift round!)
```

Round 4: Picks card “1”

```
Hand: [2, 4, 5, 6 | 1, 3]
                  ↑
              picking up


     "Oh boy, 1 needs to go all the way to the front!"
👀 SCAN: 6>1? Yes! 5>1? Yes! 4>1? Yes! 2>1? Yes!
🔄 SHIFT: [_, 2, 4, 5, 6 | 3] (everybody moves!)
📍 INSERT: [1, 2, 4, 5, 6 | 3]

Result: [1, 2, 4, 5, 6 | 3]
        ✅✅✅✅✅   ←(lots of work but perfect!)
```

Insertion Sort – Strategy

[5, 2, 4, 6, 1, 3]

Round 5: Picks card "3"

```
Hand: [1, 2, 4, 5, 6 | 3]
                     ↑
                  picking up

     "Where does 3 fit?"
👀 SCAN: 6>3? Yes! 5>3? Yes! 4>3? Yes! 2>3? No!
🔄 SHIFT: [1, 2, _, 4, 5, 6]
📍 INSERT: [1, 2, 3, 4, 5, 6]

🏆 FINAL: [1, 2, 3, 4, 5, 6] - Perfect!
```

Insertion Sort = 🃏📍 (Card player finding perfect spot)

- 🃏 PICK the next card (current element)
- 👀 SCAN backwards through sorted hand
- 🔄 SHIFT cards right to make room
- 📍 INSERT in perfect position

Pick up a card, find its spot, slide others over, place perfectly!

Insertion Sort - Code

```
3     int deck[] = {5,2,4,6,1,3};
4     int sizeOfDeck = deck.length;
5     for(int cardIndex = 1 ; cardIndex < sizeOfDeck ; cardIndex++){
6         int currentCard = deck[cardIndex]; //current card to be compared
7         int position = cardIndex - 1; //index of the previous card
8         while(position >= 0 && deck[position] > currentCard){
9             deck[position + 1] = deck[position]; //shifting to the right
10            position = position - 1; //decrementing to compare with previous
11        }
12        //placing current card at its correct position
13        deck[position + 1] = currentCard;
14    }
```

Note – cardIndex starts from 1

Insertion Sort - Performance

- 🏆 Best Case - O(n) – When cards are already sorted, lightning fast !
  - Scenario: [1, 2, 3, 4, 5, 6]
  - Just n-1 comparisons, zero shifts!
  - Each card is already in perfect position!
- 😐 Average Case - O(n^2) - Random deck... about half the cards need shifting
- 😱 Worst Case - O(n^2)
  - Scenario: [6, 5, 4, 3, 2, 1]
  - Every card needs maximum shifting
  - Total shifts: 1+2+3+...+(n-1) = n(n-1)/2

Ultimate Anchors

- 🎭 **The Family Traits:**
  - **Bubble**: "Lazy but systematic - only talks to neighbors"
  - **Selection**: "Perfectionist hunter - finds the absolute best(min)"
  - **Insertion**: "Organized card player - perfect placement every time"

⚡ Lightning Interview Responses

- **Q: "Explain the difference between all three" A**: *"Bubble compares adjacent elements and bubbles up. Selection finds the global minimum and places it correctly. Insertion takes each element and inserts it into its perfect position in the already sorted portion."*
- **Q: "Which is best for nearly sorted data?" A**: *"Insertion Sort! It's super adaptive and can run in O(n) time when data is almost sorted, while the others struggle with $O(n^2)$."*
- **Q: "Which is most stable?" A**: *"Bubble and Insertion are stable - they maintain relative order of equal elements. Selection is unstable."*
- **Q: "Which does fewest swaps?" A**: *"Selection Sort - only n swaps maximum. Bubble and Insertion can do up to n^2."*

The Three Sorting Siblings

Aspect	Bubble	Selection	Insertion
Core Action	Compare neighbours	Find global minimum	Insert perfectly
Direction	Forward bubbling →	Omnidirectional scan 🔄	Backward insertion
Best Case	O(n) sorted data	O(n²) always	O(n) sorted data
Worst Case	O(n²) 😵	O(n²) 😵	O(n²) 😵
Space	O(1)	O(1)	O(1)
Stability	Stable ✅	Unstable ❌	Stable ✅

Bubble Sort = adaptive, stable, but swap-heavy.
Selection Sort = fewest swaps but never adaptive.
Insertion Sort = best choice for nearly sorted data.

Merge 2 ‘sorted’ arrays

```
int[] arr1 = {1, 3, 5};
int[] arr2 = {2, 4, 6};
int[] mergedArray = mergeArrays(arr1, arr2);
```

mergedArray = [1, 2, 3, 4, 5, 6]

Steps for merging 2 sorted arrays

- Initial Setup

```
arr1 = [1, 3, 5]    (length = 3)
arr2 = [2, 4, 6]    (length = 3)

mergedArray = [_, _, _, _, _, _]    (length = 6)
```

Three Pointers:

- `i` = 0 (points to arr1)
- `j` = 0 (points to arr2)
- `k` = 0 (points to mergedArray)

Steps for merging 2 sorted arrays

Step 1 : Compare arr1[0] vs. arr2[0]

```
arr1 = [1, 3, 5]     i=0 ↑
arr2 = [2, 4, 6]     j=0 ↑

Compare: 1 vs 2
Result: 1 < 2, so take 1

mergedArray = [1, _, _, _, _, _]    k=0 ↑
```

Action : Copy arr1[0] to mergedArray[0], increment i and k

Step 2 : Compare arr1[1] vs. arr2[0]

```
arr1 = [1, 3, 5]        i=1 ↑
arr2 = [2, 4, 6]     j=0 ↑

Compare: 3 vs 2
Result: 3 > 2, so take 2

mergedArray = [1, 2, _, _, _, _]       k=1 ↑
```

Action : Copy arr2[0] to mergedArray[1], increment j and k

Steps for merging 2 sorted arrays

Step 3 : Compare arr1[1] vs. arr2[1]

```
arr1 = [1, 3, 5]        i=1 ↑
arr2 = [2, 4, 6]        j=1 ↑

Compare: 3 vs 4
Result: 3 < 4, so take 3

mergedArray = [1, 2, 3, _, _, _]          k=2 ↑
```

Action : Copy arr1[1] to mergedArray[2], increment i and k

Step 4 : Compare arr1[2] vs. arr2[1]

```
arr1 = [1, 3, 5]           i=2 ↑
arr2 = [2, 4, 6]        j=1 ↑

Compare: 5 vs 4
Result: 5 > 4, so take 4

mergedArray = [1, 2, 3, 4, _, _]             k=3 ↑
```

Action : Copy arr2[1] to mergedArray[3], increment j and k

Steps for merging 2 sorted arrays

Step 5 : Compare arr1[2] vs. arr2[2]

```
arr1 = [1, 3, 5]          i=2 ↑
arr2 = [2, 4, 6]          j=2 ↑

Compare: 5 vs 6
Result: 5 < 6, so take 5

mergedArray = [1, 2, 3, 4, 5, _]              k=4 ↑
```

Action : Copy arr1[2] to mergedArray[4], increment i and k

Step 6 : arr1 is exhausted, copy remaining from arr2

```
arr1 = [1, 3, 5]             i=3 (out of bounds)
arr2 = [2, 4, 6]          j=2 ↑

Only arr2[2] = 6 remains

mergedArray = [1, 2, 3, 4, 5, 6]                    k=5 ↑
```

Action : Copy arr2[2] to mergedArray[5], increment j and k

mergedArray = [1, 2 , 3 , 4 , 5 , 6]

Merging 2 sorted arrays : Key points

- Need 3 pointers
  - i tracks current position in arr1
  - j tracks current position in arr2
  - k tracks current position in mergedArray

```
arr1 = [1, 3, 5]     i=0 ↑
arr2 = [2, 4, 6]     j=0 ↑

Compare: 1 vs 2
Result: 1 < 2, so take 1

mergedArray = [1, _, _, _, _, _]    k=0 ↑
```

- Main Logic:
  - **Compare** elements at arr1[i] and arr2[j]
  - **Copy** the smaller element to mergedArray[k]
  - **Advance** the pointer of the array from which we copied
  - **Always advance** k after each copy
  - **Handle remaining** elements when one array is exhausted

Merging 2 sorted arrays : Visual Summary

```
Initial:  arr1=[1,3,5]  arr2=[2,4,6]  merged=[_,_,_,_,_,_]
Step 1:   1<2 → copy 1   merged=[1,_,_,_,_,_]
Step 2:   3>2 → copy 2   merged=[1,2,_,_,_,_]
Step 3:   3<4 → copy 3   merged=[1,2,3,_,_,_]
Step 4:   5>4 → copy 4   merged=[1,2,3,4,_,_]
Step 5:   5<6 → copy 5   merged=[1,2,3,4,5,_]
Step 6:   arr1 done → copy 6   merged=[1,2,3,4,5,6] ✓
```